Computer Architecture and Operating Systems
COSC 22073/BECS 22233

Lesson 08: Introduction to Processes

Department of Statistics & Computer Science
Faculty of Science
University of Kelaniya

What is a Process?

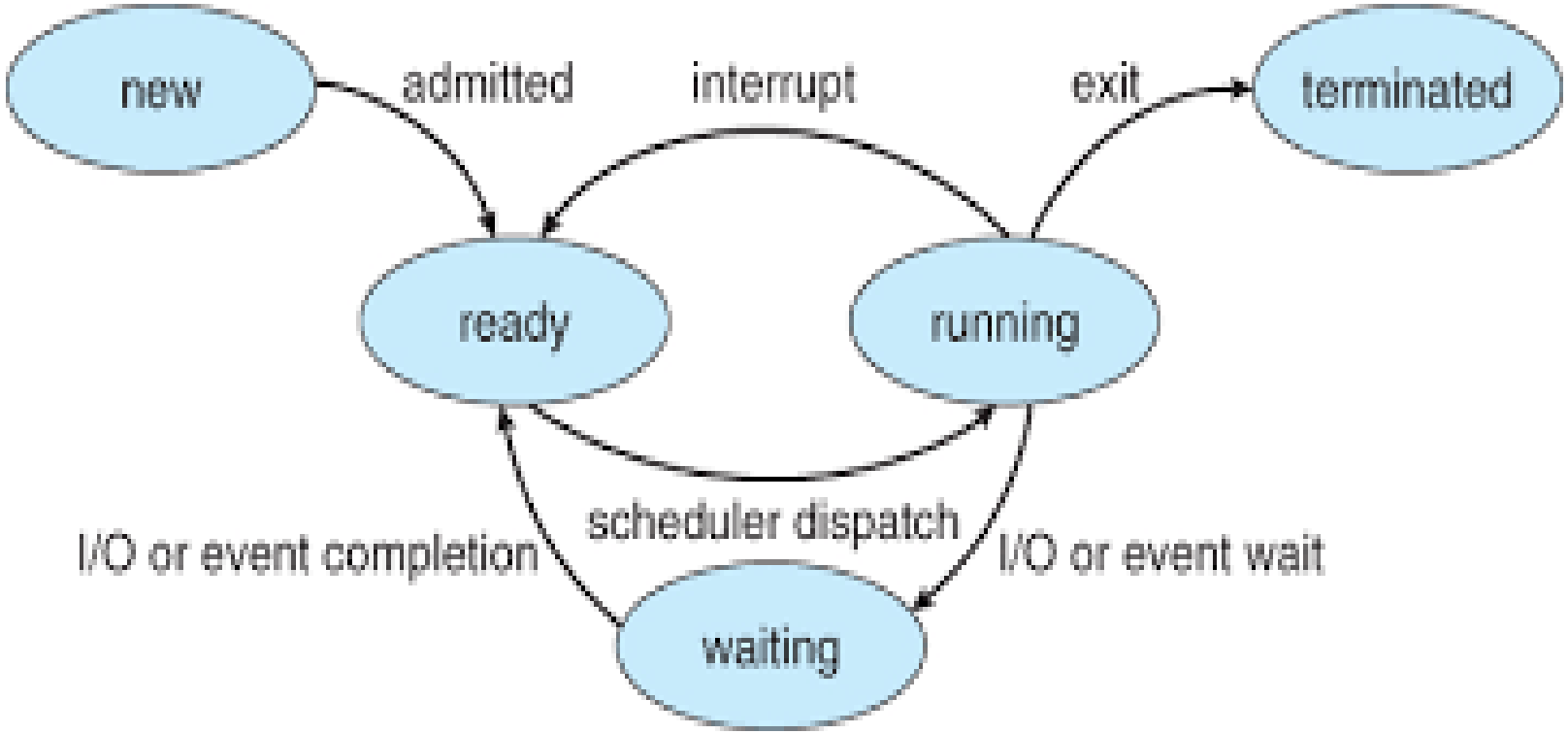
- An instance of a computer program in execution.
- Sometimes called a job or task.
- A “Process” is the name we give to a program that is running in memory.
  - So a “Program” is the compiled executable and,
  - A “Process” is the running executable with the execution state.

Process States

- During the lifecycle, a process changes state multiple times.

- Process State
  - **New**: Process is being created by the OS
  - **Ready**: Process is waiting to be assigned to a CPU core
  - **Running**: Process is executing on a CPU core
  - **Waiting**: Process is waiting for some event
  - **Terminated**: Process has finished execution

Process States

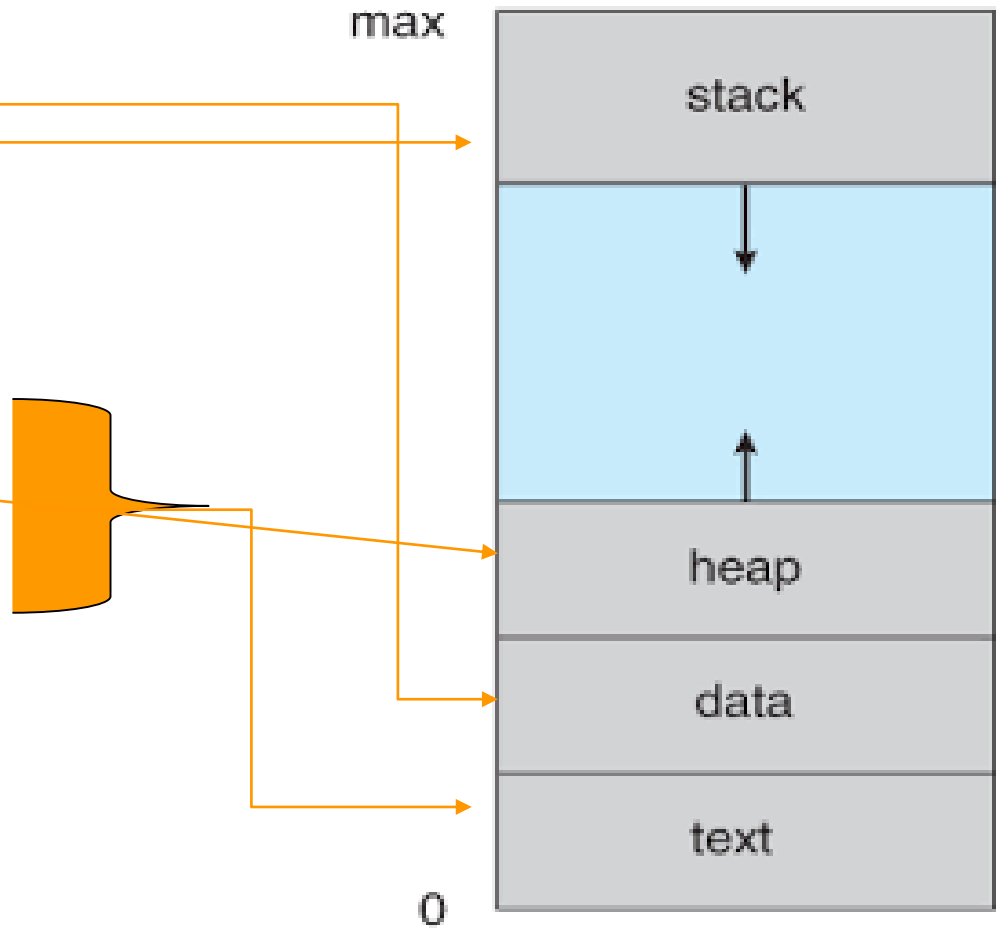


A Process Comprises of:

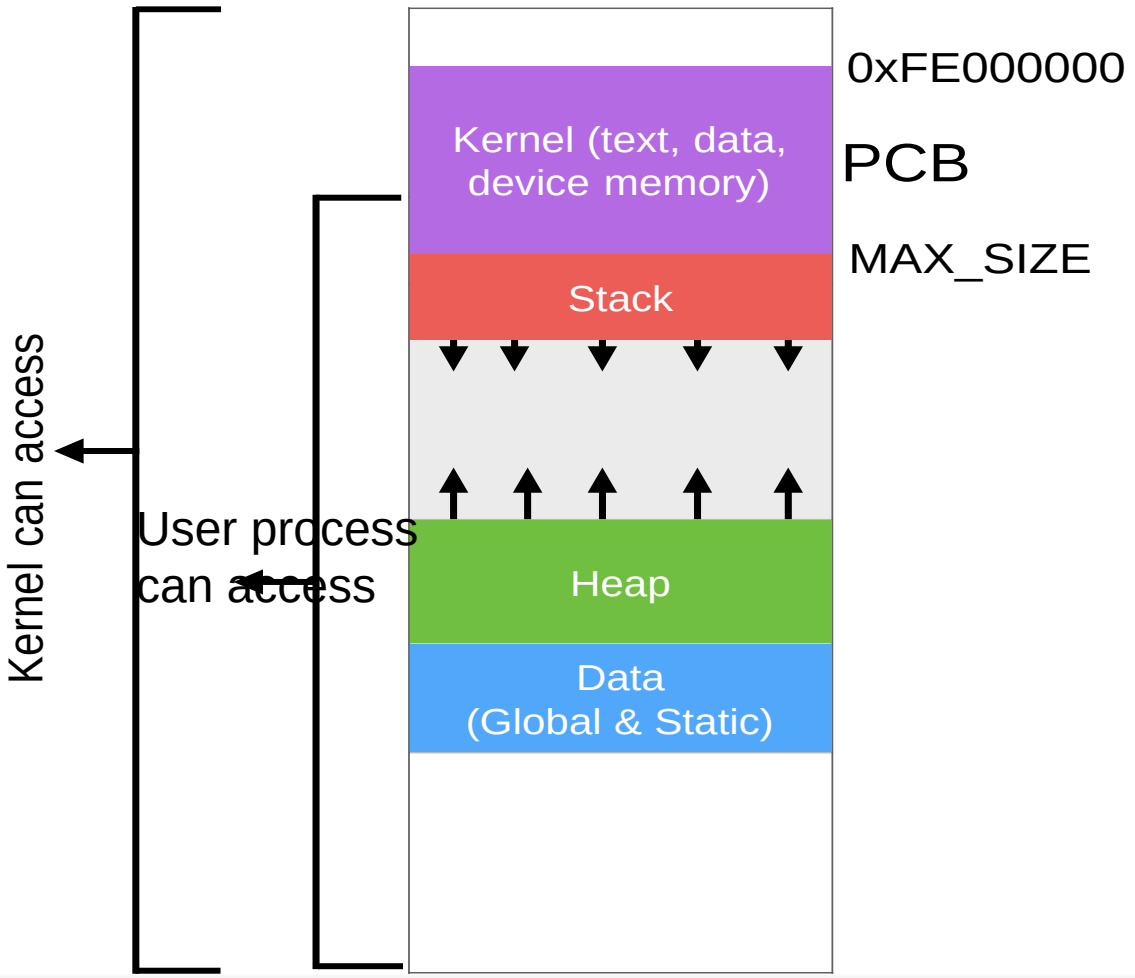
- Executable instructions
- Stack
- Heap
- State in the OS (in kernel)
  - State contains registers, a list of open files, related processes, etc.

Process Memory Map

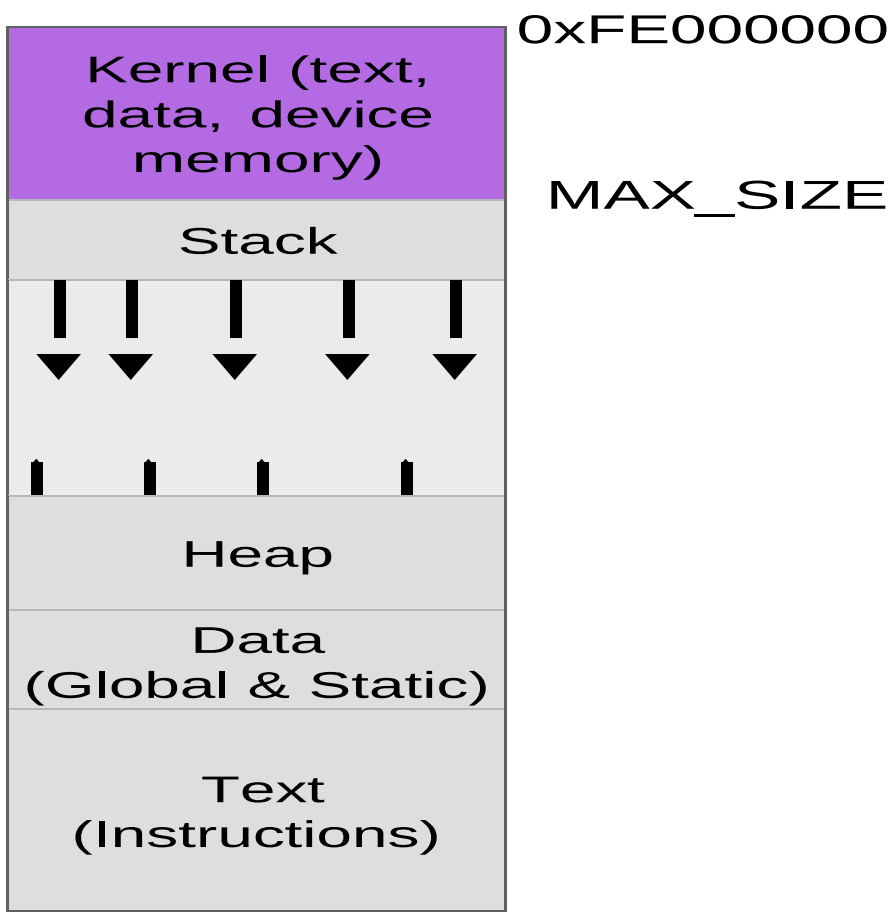
```c
#include <stdio.h>
#include <stdlib.h>

int calls;

int fact(int a);

void main()
{
        int n,*m;

        scanf("%d",&n);
        m = malloc(sizeof(int));
        *m = fact(n);
        printf("Factorial(%d) is %d\n",n,*m);
        free(m);
}

int fact(int a)
{
        calls++;
        if ( a == 1)
                return 1;
        else
                return a*fact(a-1);
}
```



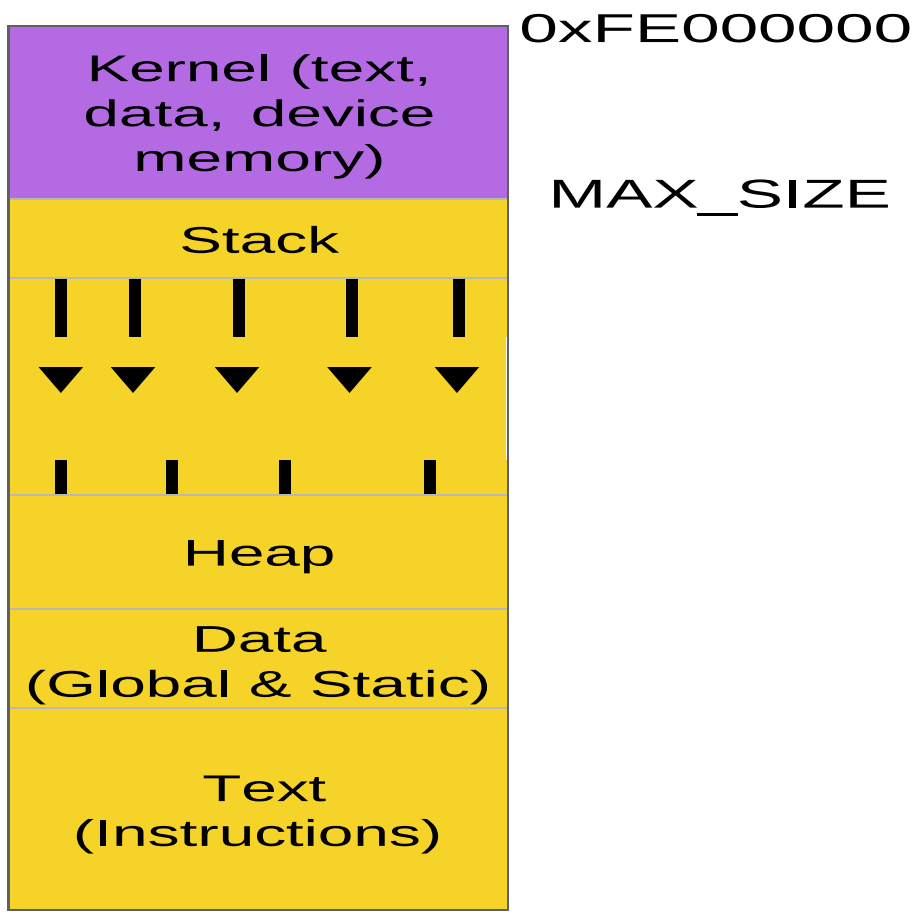
Beyond the Process Memory Map



Multiple processes in the OS



Process 0

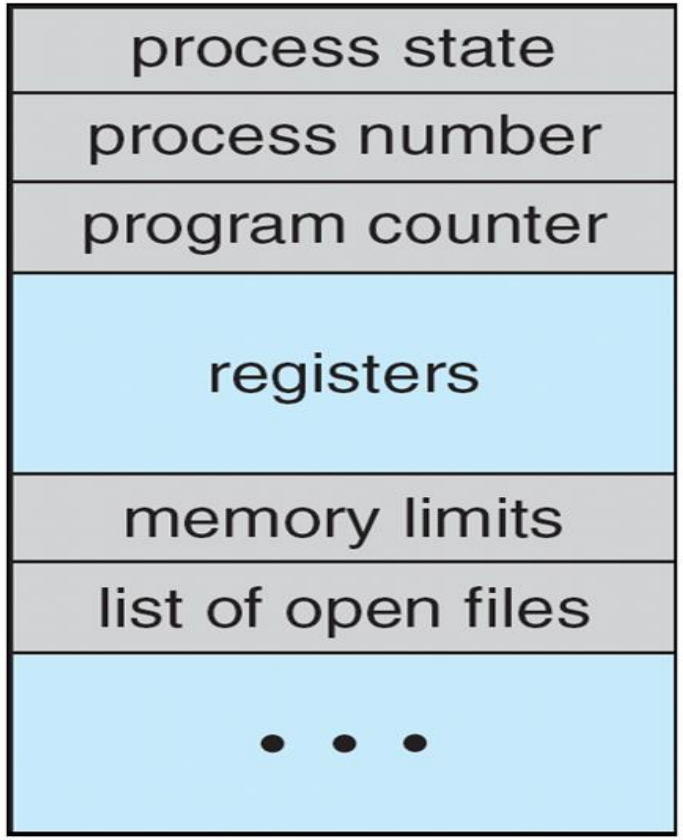


Process 1

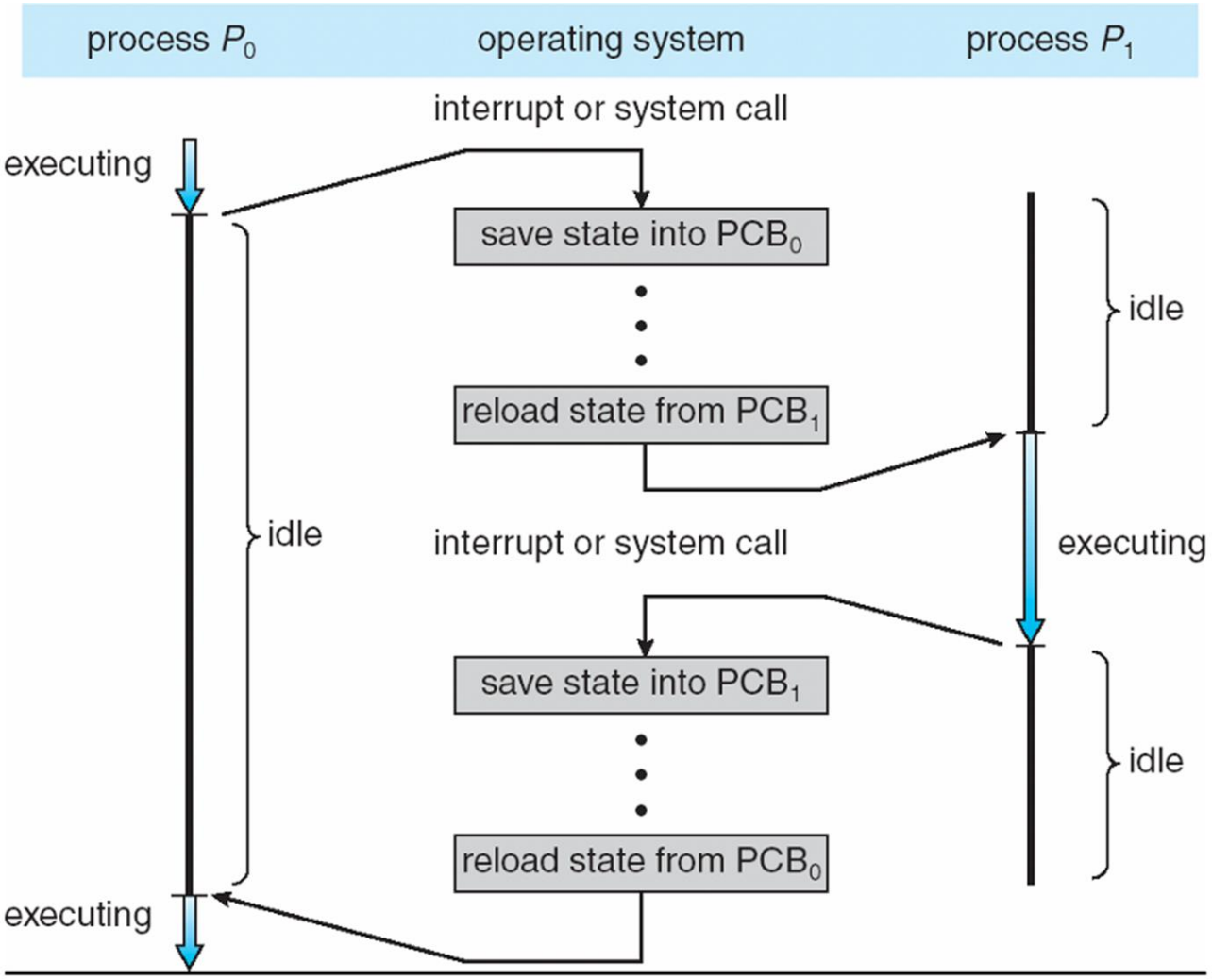
Process Control Block (PCB)

Information associated with each process (also called **task control block**)

- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files



CPU Switch from Process to Process

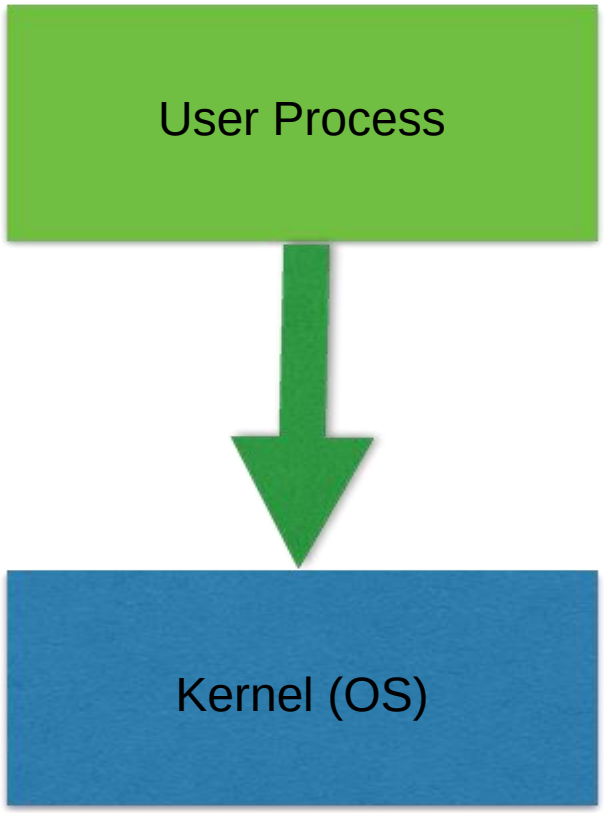


Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once

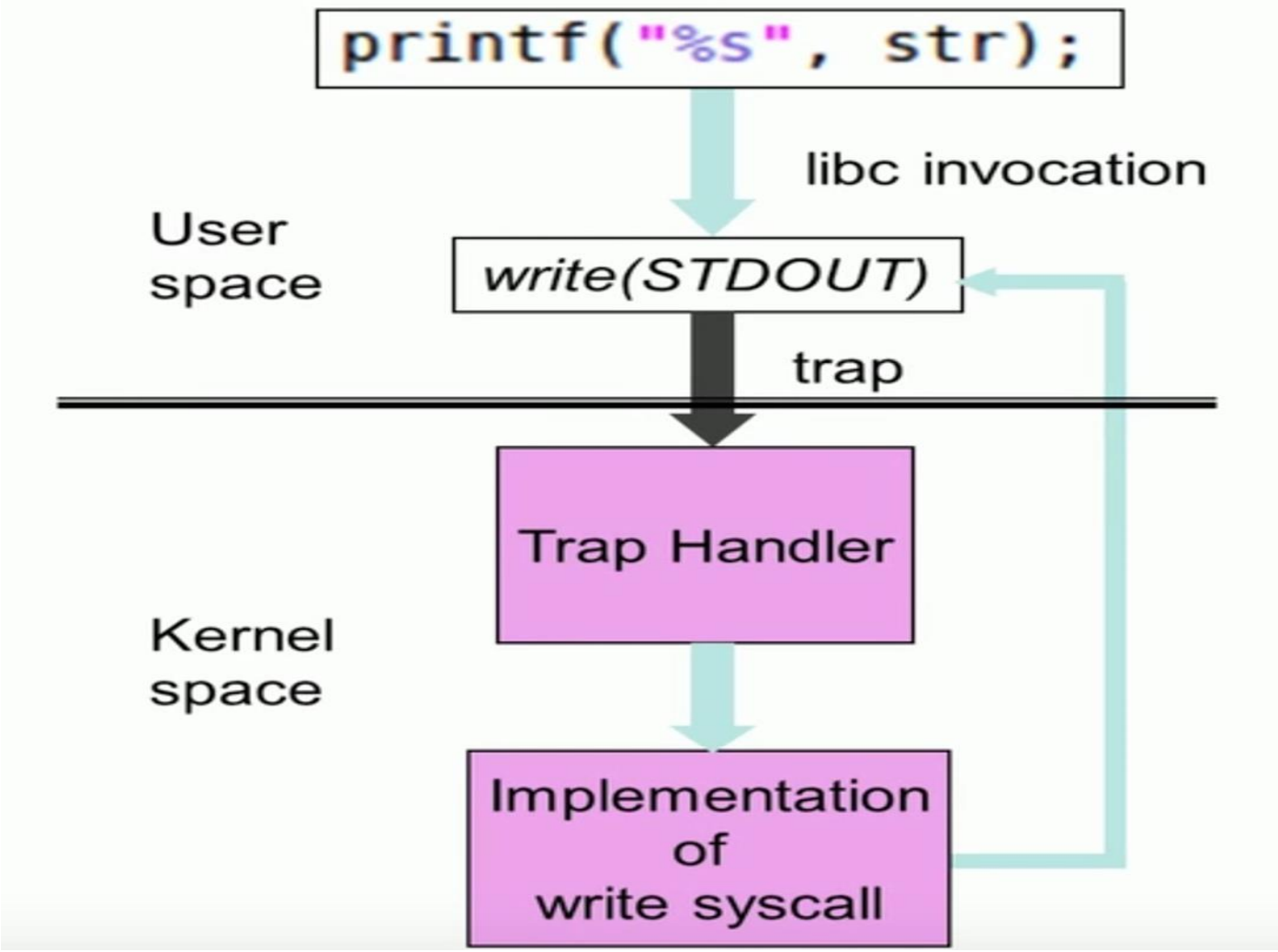
How does the user program invoke the OS?

- User processes communicate with the OS using the system call.

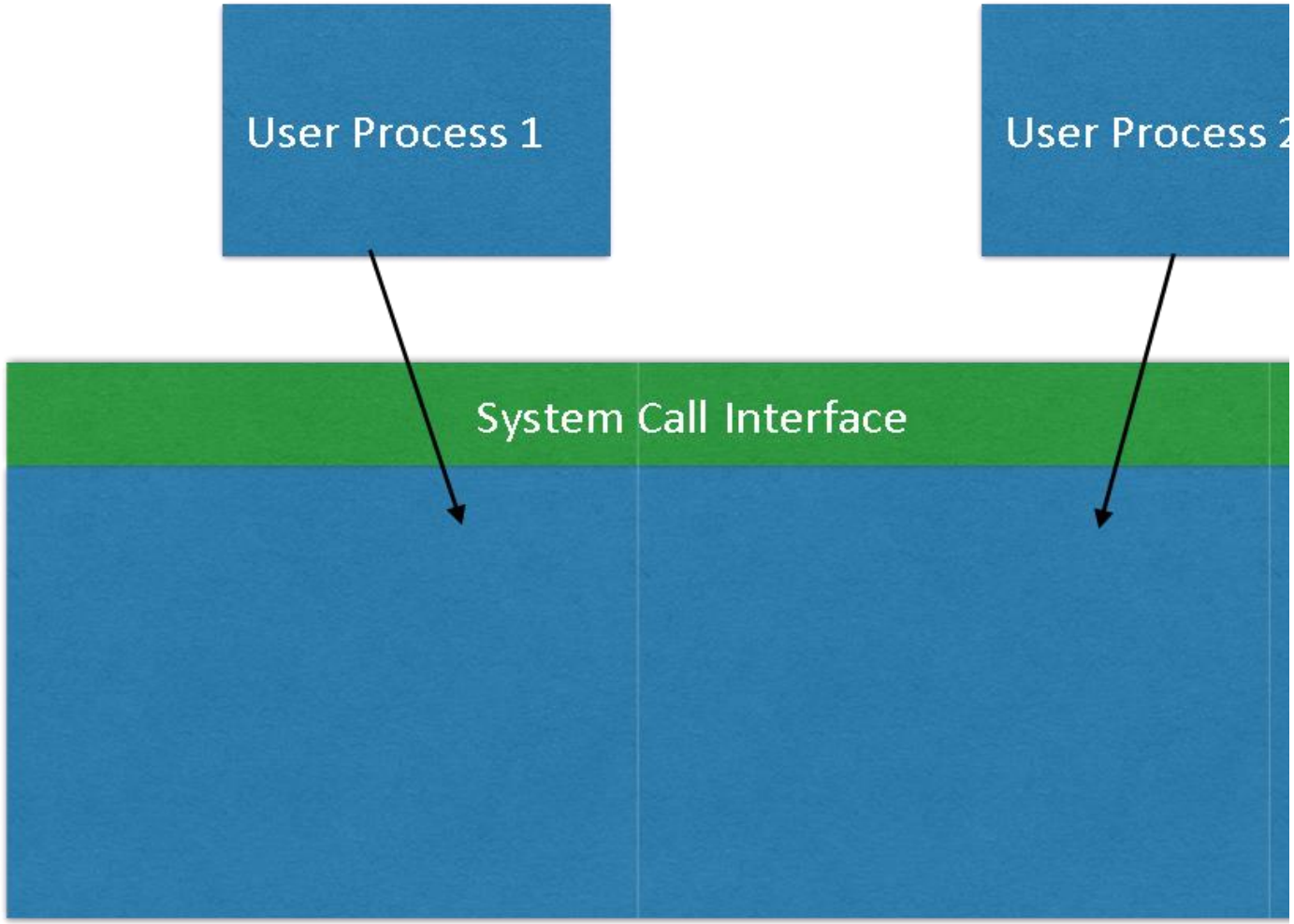


System Call

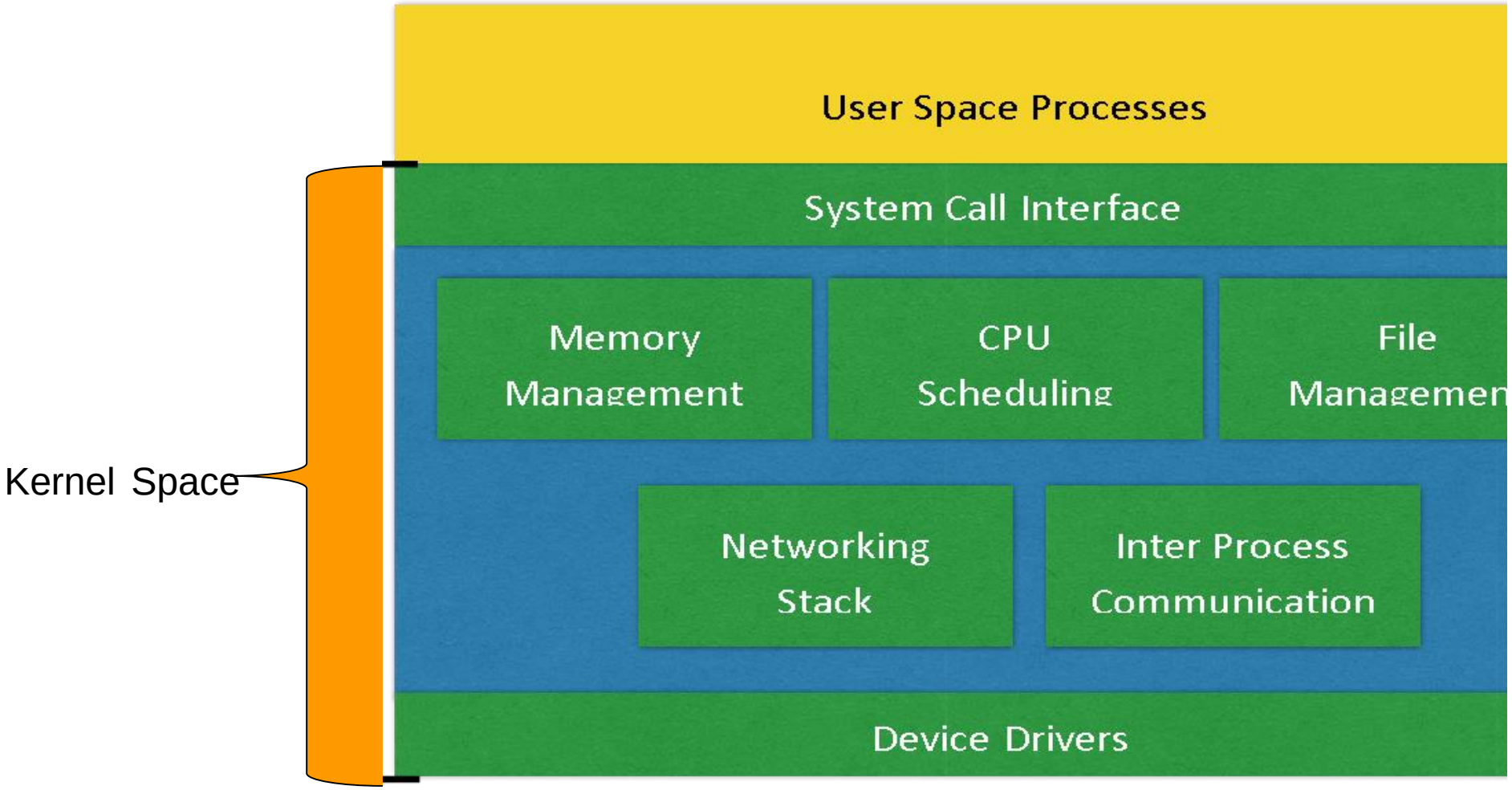
Example System Call



How the system calls are supported?



What is in the OS?



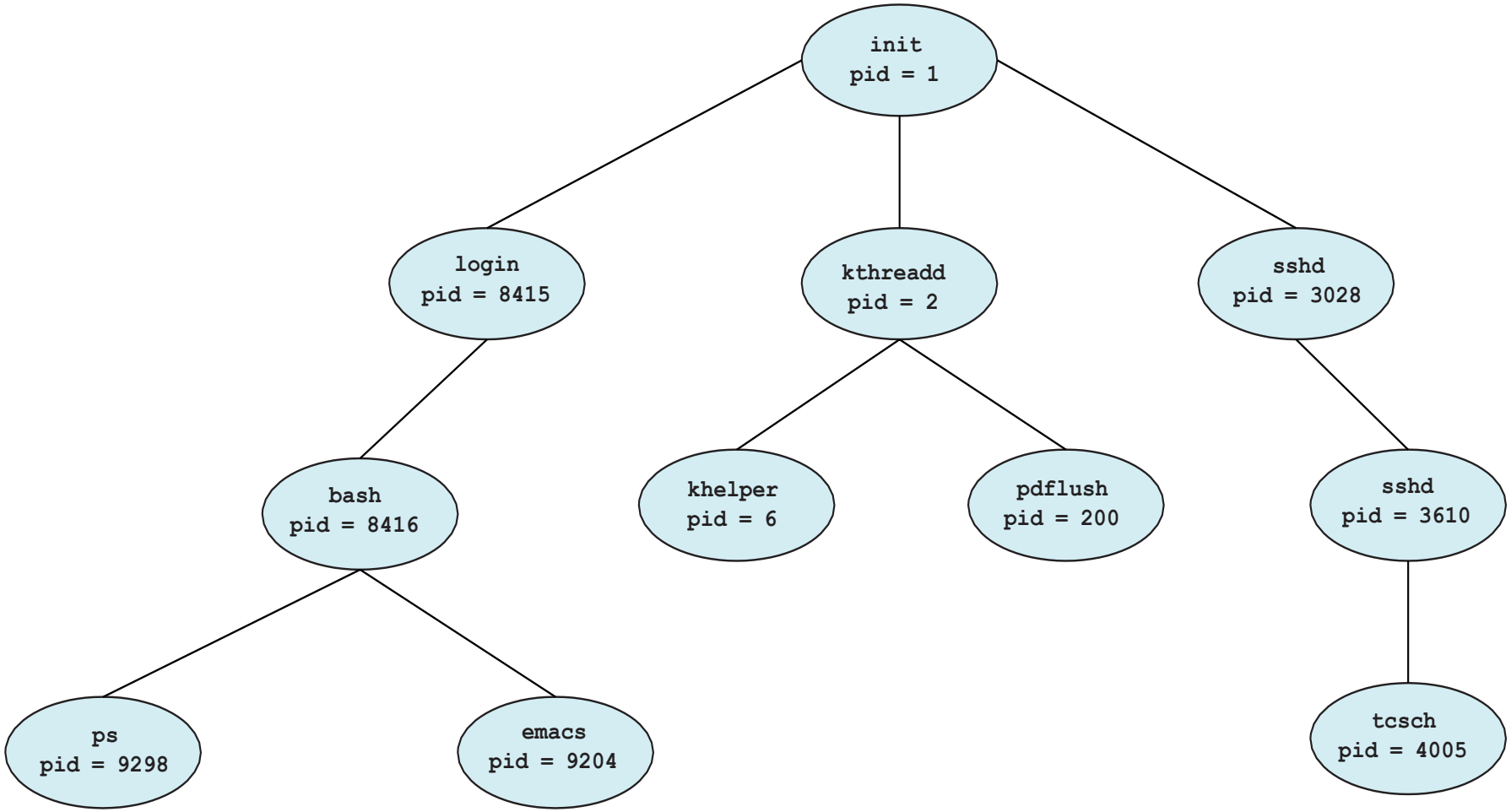
Process Creation

- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier** (**pid**)
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate

Process Creation

- Note: UNIX process model (Linux/OS X)
- All processes are created from a parent process, except for a special price called init on Linux (launched on OS)
- init (launched)
  - Process created by the OS kernel
  - Must remain alive for the entire time the system is up and running, otherwise kernel panic occurs.
  - All other processes on the system are descendants.

A Tree of Processes in Linux



Process Scheduling

- Maximize CPU use, quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
  - **Job queue** – set of all processes in the system
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Device queues** – set of processes waiting for an I/O device
  - Processes migrate among the various queues

Schedulers

- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
  - Sometimes the only scheduler in a system
  - Short-term scheduler is invoked frequently (milliseconds) ⇒ (must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
  - Long-term scheduler is invoked infrequently (seconds, minutes) ⇒ (may be slow)
  - The long-term scheduler controls the **degree of multiprogramming**

Process Termination

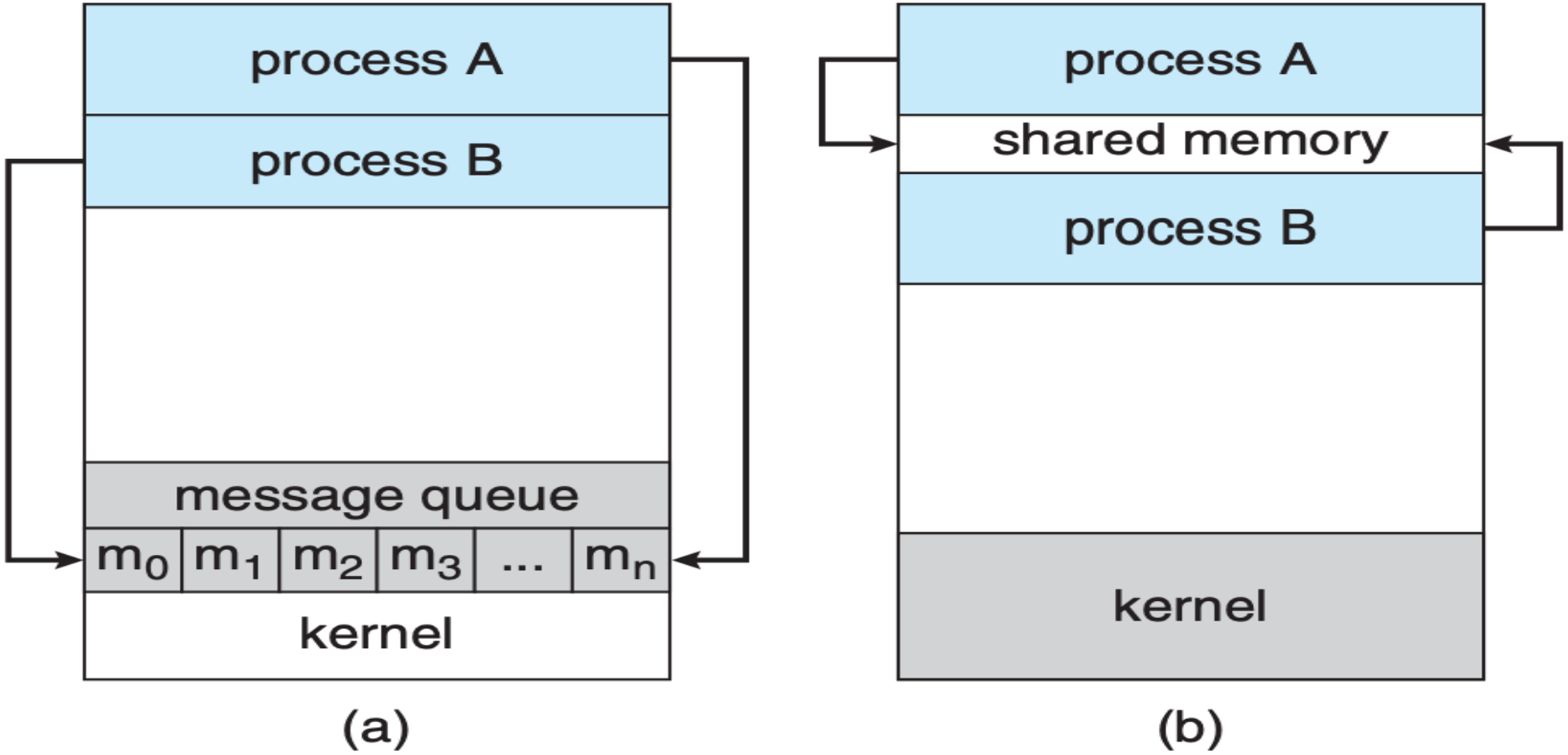
- Process executes last statement and then asks the operating system to delete it using the **`exit()`** system call.
  - Returns status data from child to parent (via **`wait()`**)
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **`abort()`** system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates

Inter-Process Communication (IPC)

- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

Communication Models

(a) Message passing. (b) shared memory.



IPC - Shared Memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.

IPC - Messages Passing

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **`send`**(*message*)
  - **`receive`**(*message*)
- The *message* size is either fixed or variable

Thank You.